Chapter 5: Architecture and Process Model of the RS-DRL-Based Self-Adaptive System

Thesis Documentation

January 5, 2026

1 Chapter Overview and Architectural Perspective

This section explains the **runtime architecture and processes** of the RS-DRL-based self-adaptive system. The emphasis is on **process-oriented refinement**, not algorithmic detail. The primary organizing principle is **process progression**—each section answers the question: "What happens next in the system, and why?" MAPE-K and RS-DRL serve as **explanatory lenses** and **secondary views**, not competing organizational structures. This section provides the chapter scope and organization overview.

The primary architectural baseline is the integration architecture shown in Figure 1 (original RS-DRL integration architecture), which serves as the reference for the process-oriented refinement presented throughout this chapter.

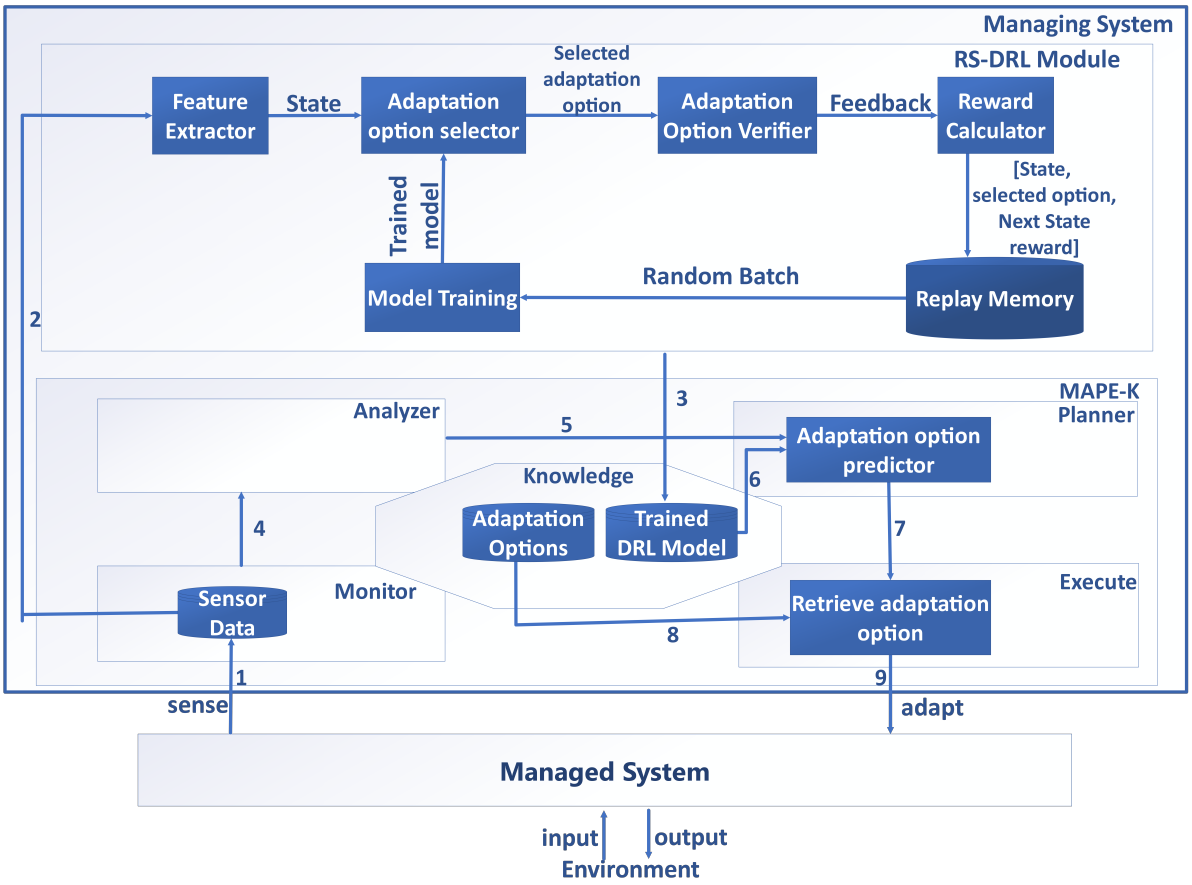

Figure 1: RS-DRL integration architecture (Figure 4). The primary architectural baseline showing the integration of RS-DRL with the MAPE-K framework for self-adaptive systems.

2 System-Level Self-Adaptation Loop

This section presents a black-box view of the system as a **continuous adaptation loop**. The system consists of two main parts: the **Managed System** and the **Managing System**. At the highest level, the system operates through a high-level **Sense–Plan–Adapt** abstraction.

This section introduces the **nine-stage adaptation flow (Stages 1–9)** used for traceability throughout the chapter. Stages 2–3 form an asynchronous learning process that runs in parallel with the runtime adaptation loop (Stages 1, 4–9). Stage 3 (asynchronous): Model update occurs independently of the runtime adaptation cycle. All subsequent sections refine **subsets of Stages 1–9**, progressively decomposing the system-level loop into focused subprocesses that answer the question: "What happens next in the system, and why?"

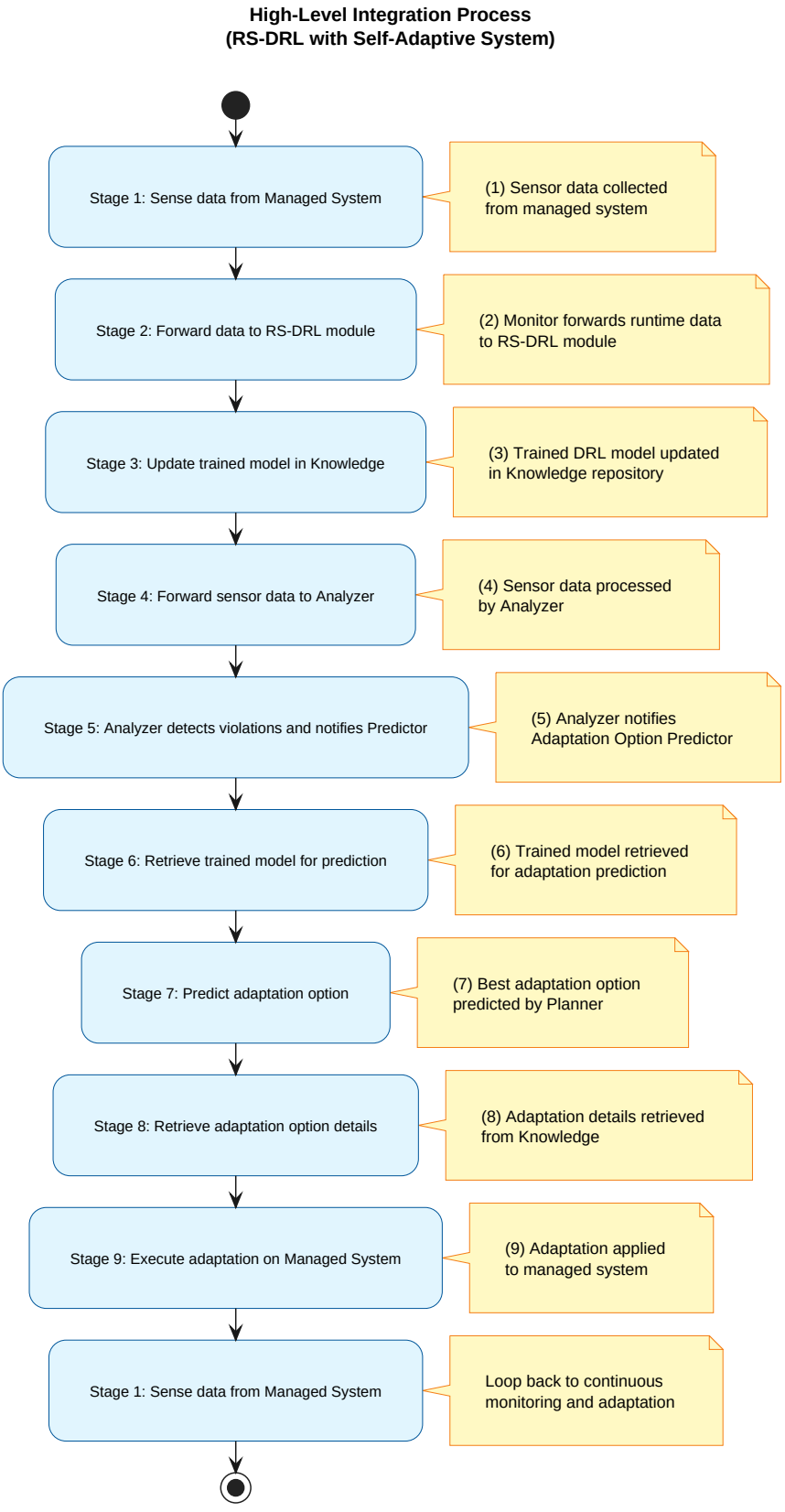

Figure 2: System-level self-adaptation process overview. This figure provides a linear, stage-based walkthrough of the RS-DRL-driven adaptation cycle. It is used throughout this chapter as the primary reference for process-oriented refinement. Stage 3 (asynchronous): Model update occurs independently of the runtime adaptation cycle.

3 Mapping the RS-DRL Architecture to MAPE-K

This section provides a **conceptual mapping** of RS-DRL processes onto the **MAPE-K loop**. This mapping serves as a **classification lens**, not a process description—it categorizes the stages introduced in Section 2 according to MAPE-K functional roles, enabling examiners familiar with MAPE-K to understand how RS-DRL integrates with established self-adaptation patterns. The role of RS-DRL is as an enhancement of the **Plan** phase. This section identifies the five MAPE-K components:

- **Monitor**: Observes the managed system and collects runtime data
- **Analyze**: Evaluates system state against adaptation goals
- **Plan**: Makes adaptation decisions using RS-DRL learned policies
- **Execute**: Applies selected adaptations to the managed system
- **Knowledge**: Stores trained models, adaptation options, and state history

This section classifies processes by **temporal nature** (continuous, event-driven, conditional, asynchronous).

The architectural realization of the process illustrated in Figure 2 is shown in Appendix A (Figure A.1), where the RS-DRL-based adaptation loop is mapped onto the MAPE-K reference model.

4 Monitoring and State Construction Process

This section details the monitoring and state construction process, which performs runtime data collection from the managed system. The process includes preprocessing and feature extraction, leading to the construction of the **state representation** used for decision-making. The constructed state information is forwarded to both analysis and learning processes. Stage 2 provides state information to the RS-DRL module for both runtime inference and asynchronous learning, without implying synchronous training during runtime operation.

Process boundary: This monitoring and state construction process ends once a validated state vector exists and has been disseminated. Everything that follows—decision-making, learning, or execution—operates on this validated state representation, maintaining a clear separation of concerns between perception (monitoring) and action (decision and execution).

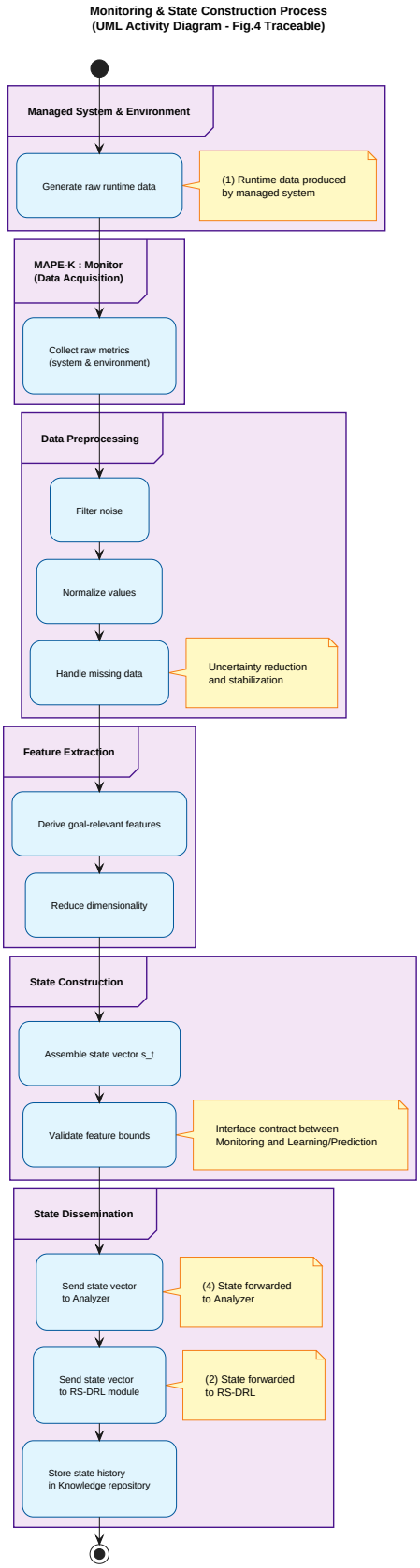

Figure 3: Monitoring and state construction: perception process transforming raw data into decision-ready state vectors. This process starts at Stage 1 (data acquisition) and terminates at Stage 4 (state dissemination to Analyzer) and Stage 2 (state forwarding to RS-DRL module). Raw runtime data is filtered, abstracted, and transformed into a validated state vector suitable for both RS-DRL learning and runtime adaptation analysis.

5 Online Adaptation Decision Process

The following diagrams refine the system-level process overview (Figure 2) by decomposing it into focused subprocesses. Each diagram represents a single responsibility at a consistent abstraction level. This decomposition avoids cognitive overload while preserving traceability through the numbered stages (1–9) and the system-level overview presented in Section 2.

This section describes the **runtime decision-making pipeline**, executed when adaptation is required. The process encompasses analysis, planning, and verification phases.

5.1 Analysis and Adaptation Triggering

This subsection covers the evaluation of system state against goals and constraints. It details the detection of goal violations or risk conditions, which triggers the planning process.

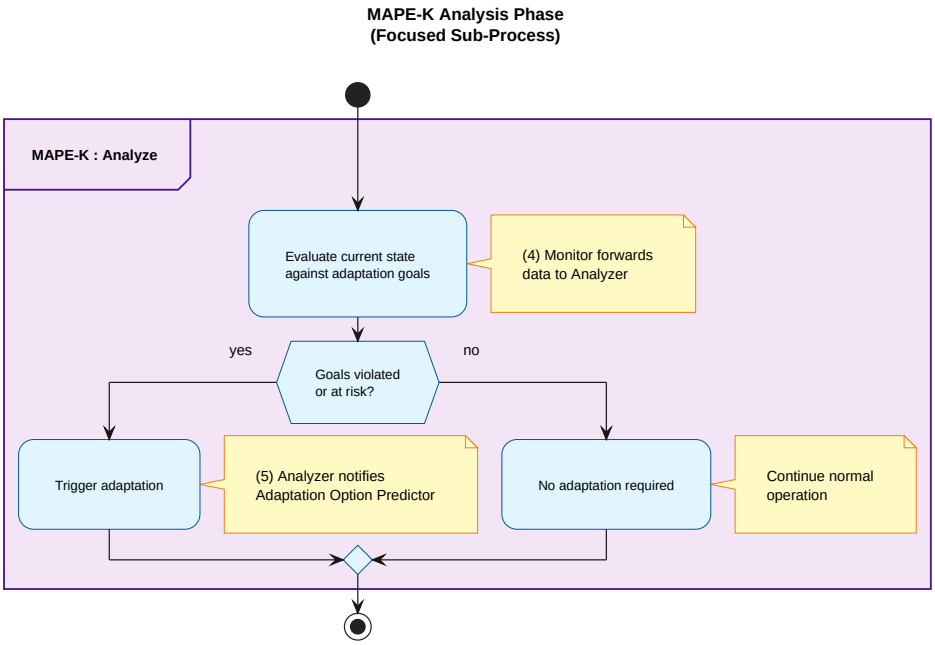

Figure 4: Analysis and adaptation triggering: when and why the system decides to adapt. This process starts at Stage 4 (state reception) and terminates at Stage 5 (notification to Predictor). The Analyzer component evaluates system state against adaptation goals, detects violations or risk conditions, and triggers the planning phase when intervention is required.

5.2 Planning and Adaptation Option Prediction

This subsection covers the retrieval of the trained RS-DRL model, the prediction of candidate adaptation options, and the selection of the most suitable option based on learned policy. The planning process completes at Stage 7 with the predicted adaptation option, which is then forwarded for verification and execution in subsequent phases.

Process boundary: This planning process is **pure inference**—it uses a pre-trained model to evaluate options but performs no learning, no replay memory sampling, and no policy updates. All training logic is strictly isolated to the asynchronous learning process (Section 8).

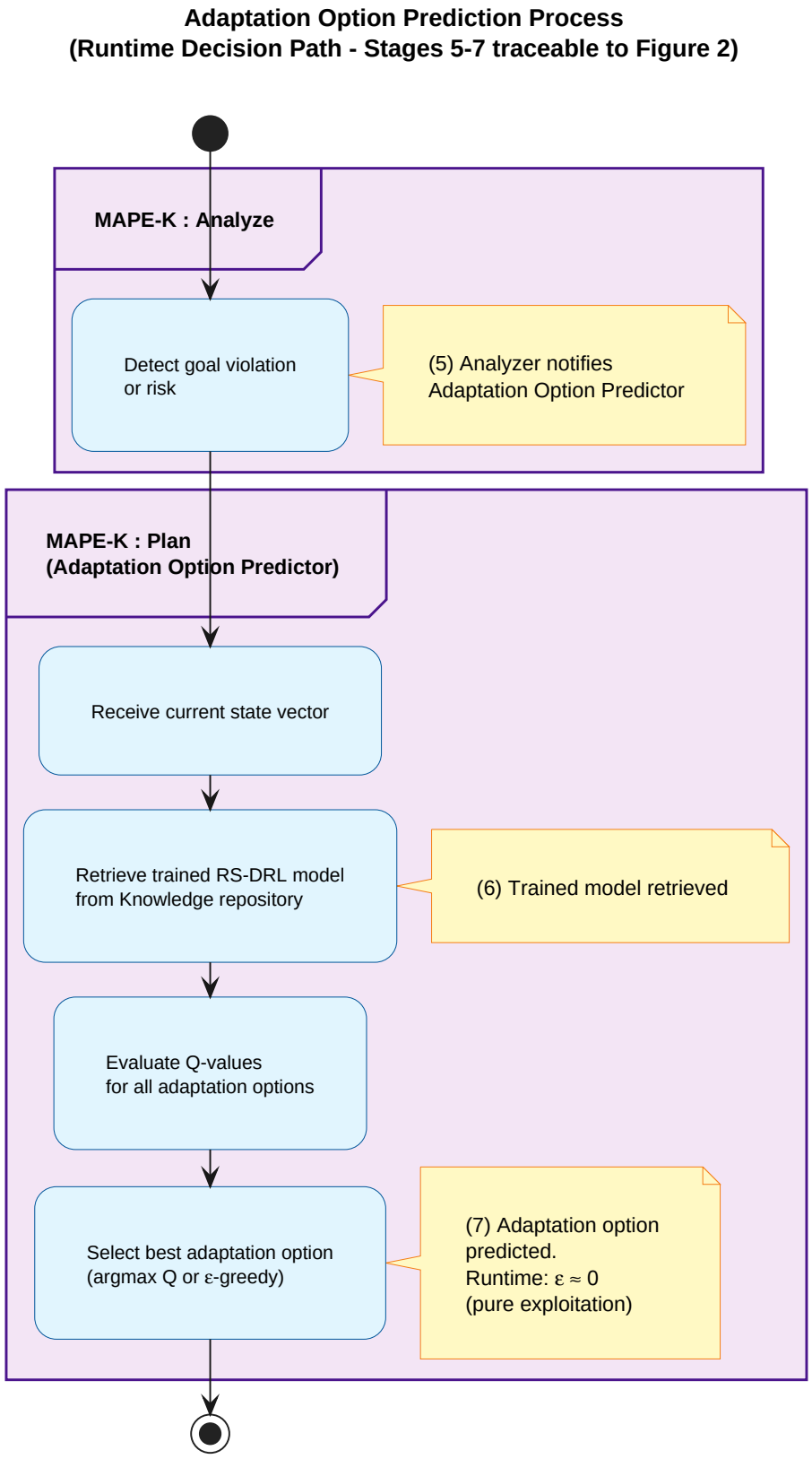

Figure 5: Runtime decision path: adaptation option prediction without learning. This process starts at Stage 5 (Analyzer notification) and terminates at Stage 7 (option prediction). Upon detection of goal violations, the trained RS-DRL model is retrieved (Stage 6) and used to predict the most suitable adaptation option through Q-value evaluation. The adaptation option predicted at Stage 7 is the final output of this process. This figure explicitly excludes training, replay memory sampling, policy updates, and execution (Stage 8) to maintain focus on runtime decision-making.

5.3 Adaptation Option Verification

This subsection covers the validation of predicted options under uncertainty. It details the use of runtime models and statistical evaluation. Verification belongs to **runtime justification**—its primary purpose during the adaptation decision process is to evaluate predicted options and provide quality estimates that justify adaptation decisions. While verification also generates reward signals that are consumed by the learning process (Section 8), reward generation is a **by-product** of the verification process, not its primary goal during runtime decision-making.

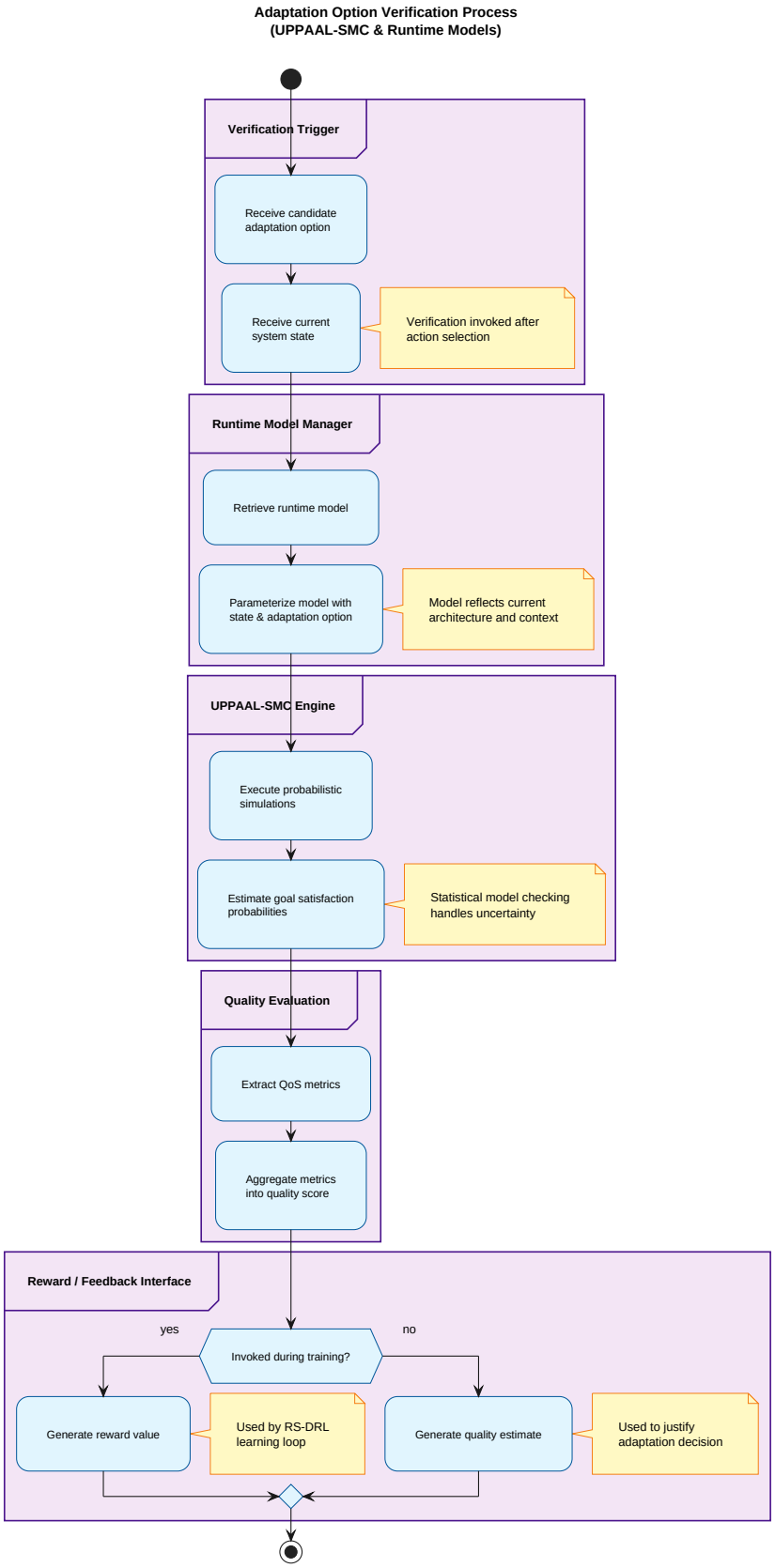

Figure 6: Uncertainty evaluation subprocess: adaptation option verification using statistical model checking. This process validates predicted options under uncertainty, using runtime models and UPPAAL-SMC to estimate goal satisfaction probabilities. The verification process extracts QoS metrics and generates two distinct outputs: **Decision feedback (runtime)** for decision justification during planning, and **Learning reward (asynchronous)** for training during learning, ensuring semantic grounding for learning-based decisions.

6 RS-DRL Internal Decision and Learning Architecture

This section corresponds to the **RS-DRL module** shown in Figure 4 of the original article, which serves as the innovation center of the self-adaptive system. While MAPE-K provides the architectural framework for self-adaptation, the RS-DRL module is what makes adaptation scalable, robust, and uncertainty-aware. The RS-DRL module is realized through the processes described in the following sections: the runtime inference path (Section 5.2 and Section 5.3), feedback generation (Section 7), and the learning path (Section 8).

The RS-DRL module encompasses three interconnected subprocesses: (1) the **runtime inference path**, which includes the Adaptation Option Selector (embedded in the Planner) and the Adaptation Option Verifier, enabling informed decision-making using learned policies; (2) the **feedback generation path**, which includes the Reward Calculator that transforms verification outcomes into reward signals; and (3) the **learning path**, which includes Replay Memory, the novel reward-shaping mechanism, and Model Training, enabling continuous policy improvement. These subprocesses collectively implement the RS-DRL module's core functionality, with the reward-shaping mechanism representing the key innovation that accelerates learning and promotes generalization across heterogeneous environments.

7 Execution and System Reconfiguration Process

This section covers the retrieval of adaptation option specifications, the application of the selected adaptation to the managed system, the effect of execution on system behavior and environment, and the reintegration into the continuous monitoring loop.

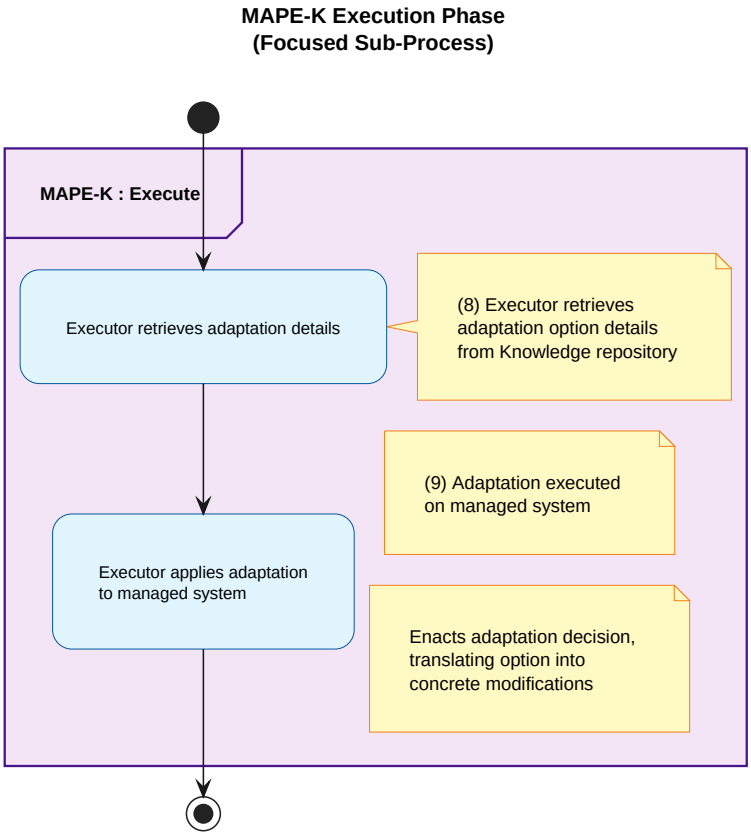

Figure 7: Execution and system reconfiguration: how decisions affect the managed system. This process starts at Stage 8 (option retrieval) and terminates at Stage 9 (adaptation application). The Executor component retrieves adaptation option specifications and applies the selected adaptation to the managed system, completing the adaptation cycle.

8 RS-DRL Feedback and Reward Calculation Process

This section covers the **generation of feedback signals** from execution and verification outcomes. This section focuses on feedback generation only—how verification outcomes and execution results are transformed into reward signals and experience tuples. The consumption of this feedback by the learning process is detailed in Section 8. This separation maintains the logical distinction between **online decision-making** and **learning feedback**.

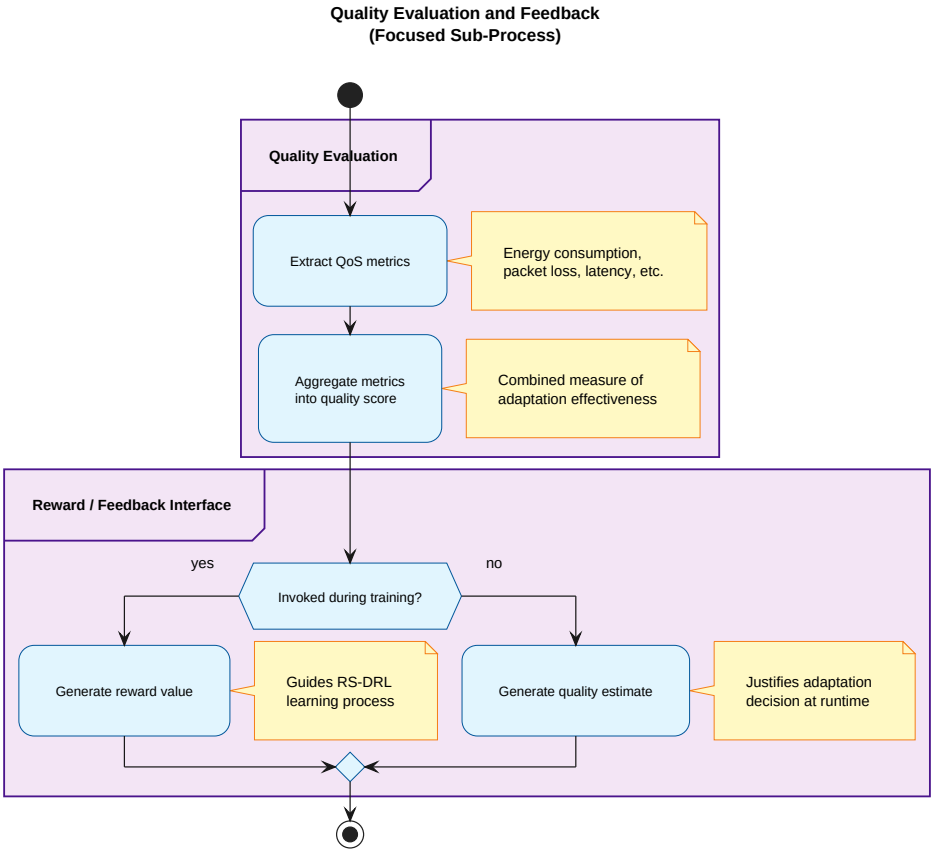

Figure 8: Feedback and reward generation: how outcomes are transformed into learning signals. This bridge process extracts QoS metrics from verification results, aggregates quality attributes, and generates two distinct outputs: **Learning reward (asynchronous)** for training, and **Decision feedback (runtime)** for decision justification. This figure explicitly excludes training steps, focusing solely on the transformation of outcomes into feedback.

9 RS-DRL Learning and Policy Improvement (Reward-Shaping Core)

This section details the asynchronous RS-DRL training process, including experience replay and reward shaping. This section focuses on **learning consumption of feedback**—how the learning process uses verification-derived rewards and experience tuples (generated in Section 7) to improve adaptation policies. It covers model update and storage in the Knowledge repository, and the availability of updated policies for future adaptation cycles.

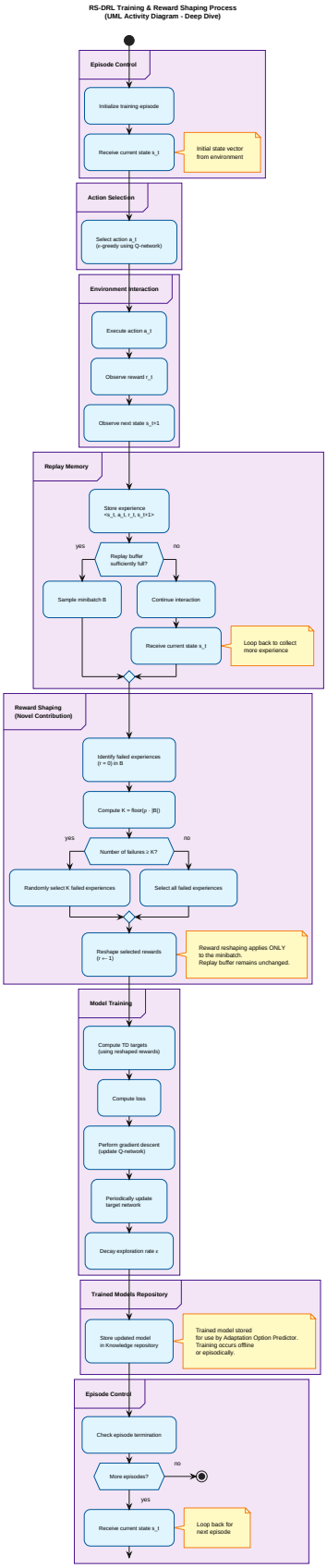

Figure 9: Asynchronous learning and policy update: RS-DRL training and reward shaping process. This is the only figure allowed to be dense, as learning is inherently algorithmic and complex. The figure details episodic interaction with the environment, experience replay, and the novel reward reshaping mechanism applied during minibatch learning. The process encompasses episode initialization, action selection, environment interaction, experience storage, replay buffer management, minibatch sampling, reward shaping, Q-network training, and model storage (Stage 3).

10 Knowledge Management and Process Coordination

This section explains knowledge management from a **coordination semantics** perspective, focusing on **who reads what and when**, rather than re-explaining storage mechanics that have already been described in previous sections. The Knowledge repository serves as shared system memory, storing trained models, adaptation options, and state history.

Knowledge coordination occurs through well-defined read-write contracts:

- **When monitoring completes** (Stage 1): State vectors are stored in Knowledge for historical reference.
- **When learning completes** (Stage 3): Trained models are written to Knowledge, making learned policies available for future runtime use.
- **When planning begins** (Stage 6): Trained models are read from Knowledge, enabling the Adaptation Option Predictor to leverage learned knowledge.
- **When execution begins** (Stage 8): Adaptation option details are read from Knowledge, providing the Executor with the specifications needed to enact adaptations.

This coordination structure ensures that all components operate on consistent information while maintaining clear separation of concerns. The Knowledge repository enables independent processes with different temporal natures (continuous monitoring, event-driven prediction, asynchronous learning) to synchronize effectively without tight coupling.

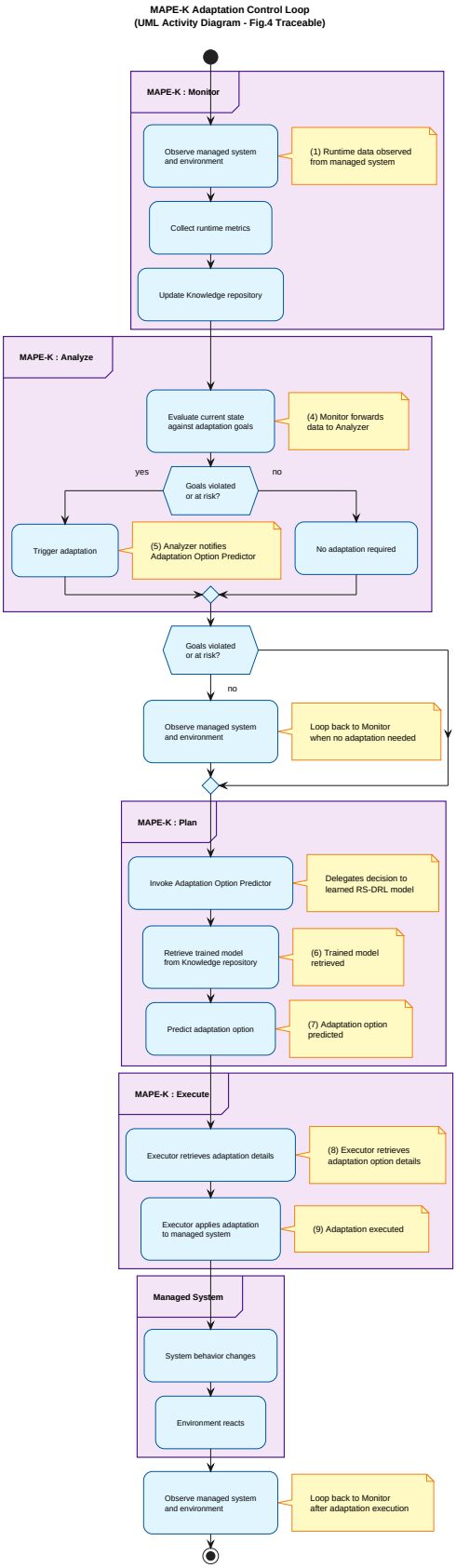

Figure 10: MAPE-K control loop summary: validation view of the adaptation architecture. This optional figure serves as a control-theoretic validation lens, illustrating how monitoring and analysis trigger adaptation, how planning delegates decision making to the RS-DRL predictor, and how execution enacts the selected adaptation. This figure complements the system-level process overview (Figure 2) by providing a MAPE-K-specific perspective for theory-oriented examiners.

11 Summary of Architectural Processes

This section provides a recap of the hierarchical process decomposition. It reinforces RS-DRL as the **dominant end-to-end adaptation process** and prepares for evaluation and experimental analysis in the next chapter.